



University
of Exeter

COMM113

Deep Learning – Week 8

Recurrent Neural Networks

Gaojie (Jay) Jin
g.jin@exeter.ac.uk

RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$

hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$

outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$

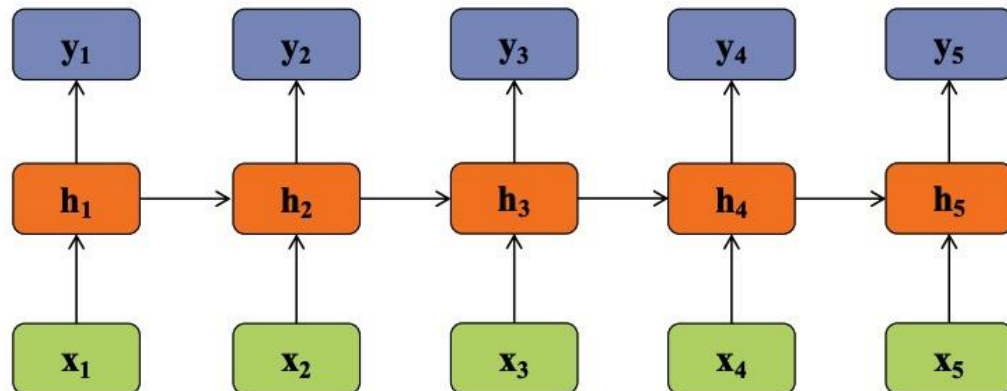
nonlinearity: $\mathcal{H}$

Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Q: Why does an RNN use the same parameters at every time step?



RNN

inputs: $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$

hidden units: $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$

outputs: $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$

nonlinearity: $\mathcal{H}$

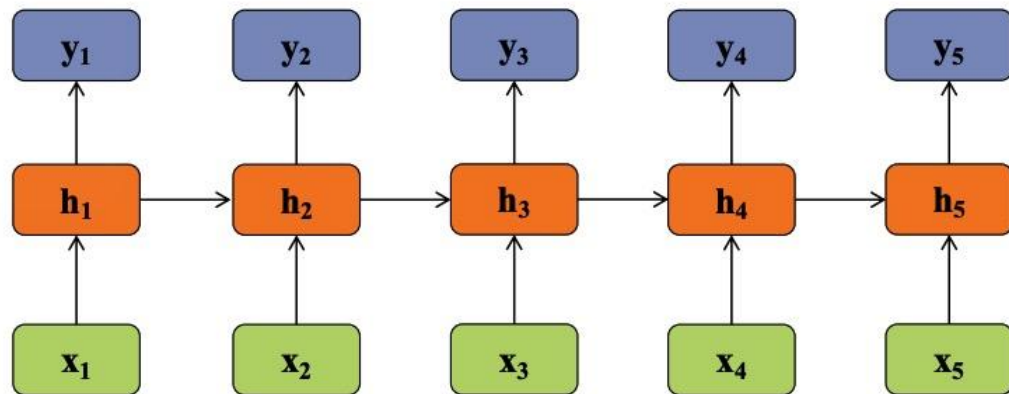
Definition of the RNN:

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = W_{hy}h_t + b_y$$

Q: Why does an RNN use the same parameters at every time step?

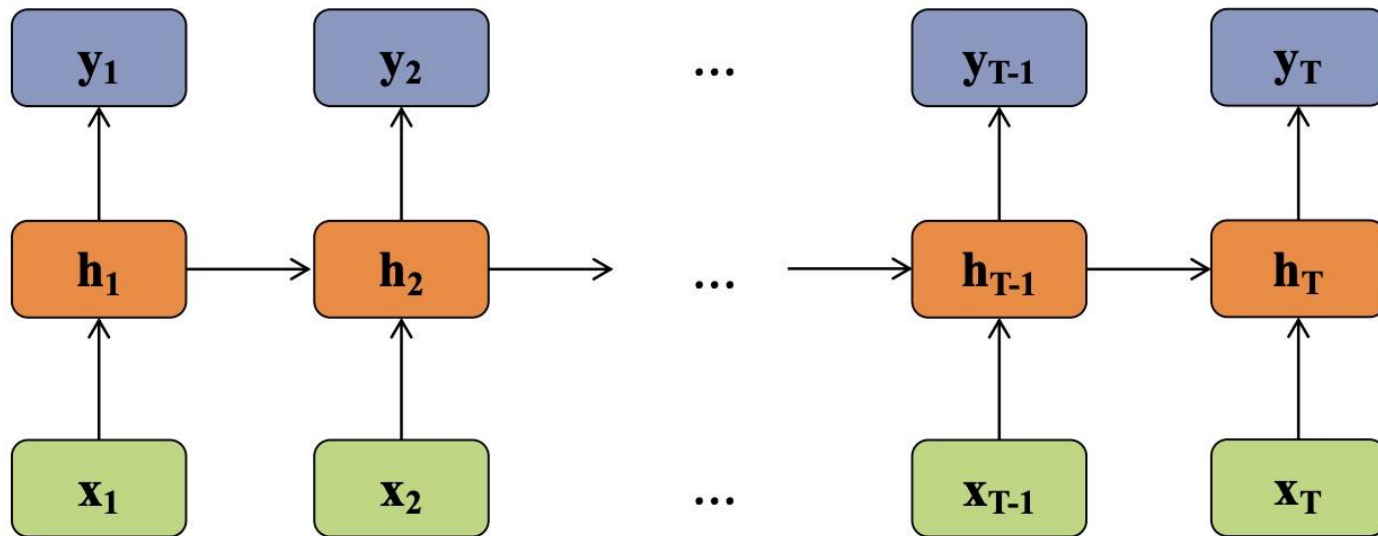
A: To model sequential patterns efficiently and reduce parameter count.



Long Short-Term Memory (LSTM)

Motivation:

- Standard RNNs have trouble learning long distance dependencies
- LSTMs address this issue



Long Short-Term Memory (LSTM)

Motivation:

- Standard RNNs have trouble learning long distance dependencies
- LSTMs address this issue

Task: Given input sequence [a, b, c, d, e], output the same sequence.

For sequences > 20 steps:

X RNN forgets earlier characters

✓ LSTM can store the sequence in its cell state

Q: Why does an RNN fail at copying long sequences?

Long Short-Term Memory (LSTM)

Motivation:

- Standard RNNs have trouble learning long distance dependencies
- LSTMs address this issue

Task: Given input sequence [a, b, c, d, e], output the same sequence.

For sequences > 20 steps:

✗ RNN forgets earlier characters

✓ LSTM can store the sequence in its cell state

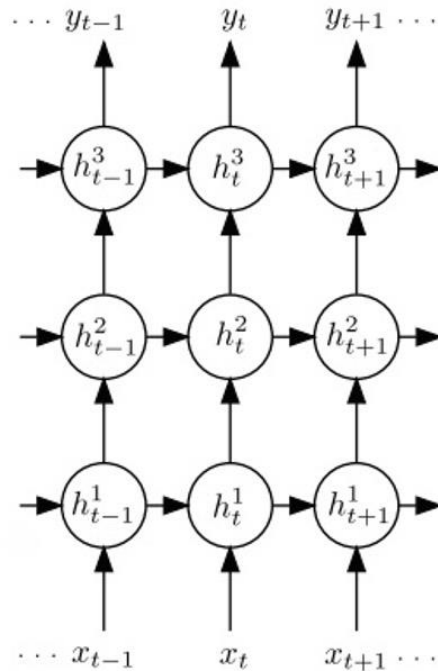
Q: Why does an RNN fail at copying long sequences?

A: Early gradients vanish, so early inputs are forgotten.

Vanishing Gradient Problem in RNNs

In Deeper RNNs

Q: Which two axes in RNNs can suffer from vanishing gradients?

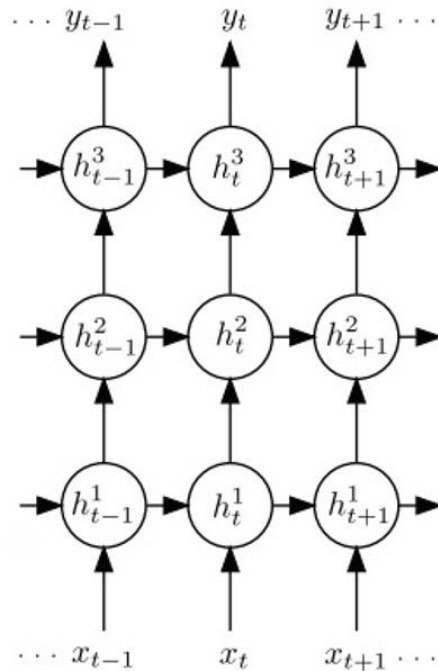


Vanishing Gradient Problem in RNNs

In Deeper RNNs

Q: Which two axes in RNNs can suffer from vanishing gradients?

A: Depth and time.

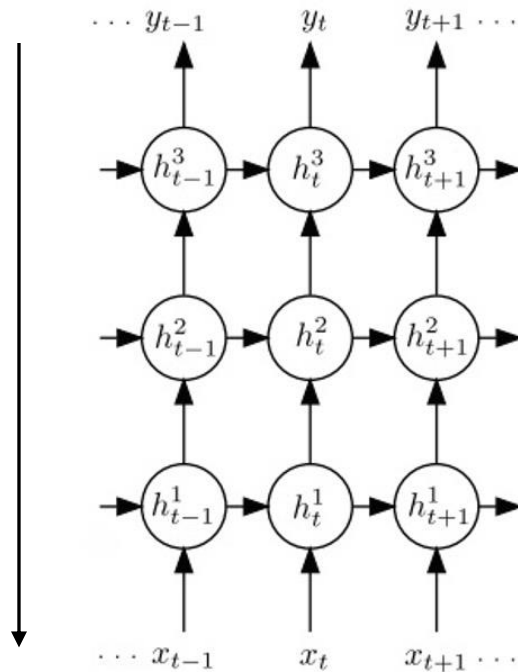


Vanishing Gradient Problem in RNNs

In Deeper RNNs (two sources of vanishing gradients)

RNNs can be deep in two dimensions: time and layers (depth). Vanishing gradients happen through both.

- Depth: backward through layers



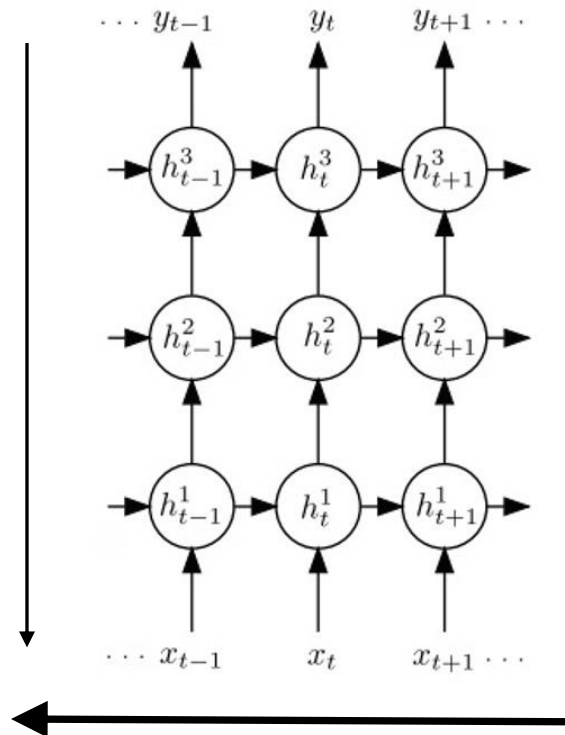
Vanishing Gradient Problem in RNNs

In Deeper RNNs (two sources of vanishing gradients)

RNNs can be deep in two dimensions: time and layers.
Vanishing gradients happen through both.

- Depth: backward through layers
- Temporal (time): backward through time steps

Q: Which type of vanishing gradient does LSTM primarily fix?



Vanishing Gradient Problem in RNNs

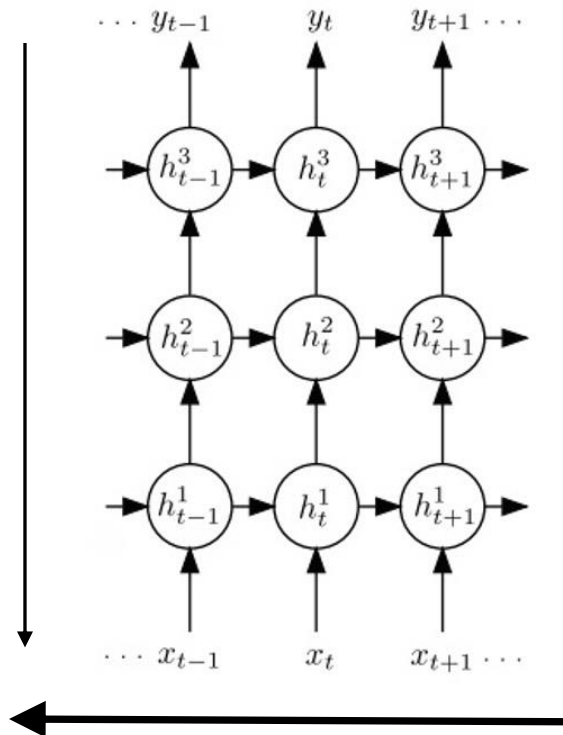
In Deeper RNNs (two sources of vanishing gradients)

RNNs can be deep in two dimensions: time and layers.
Vanishing gradients happen through both.

- Depth: backward through layers
- Temporal (time): backward through time steps

Q: Which type of vanishing gradient does LSTM primarily fix?

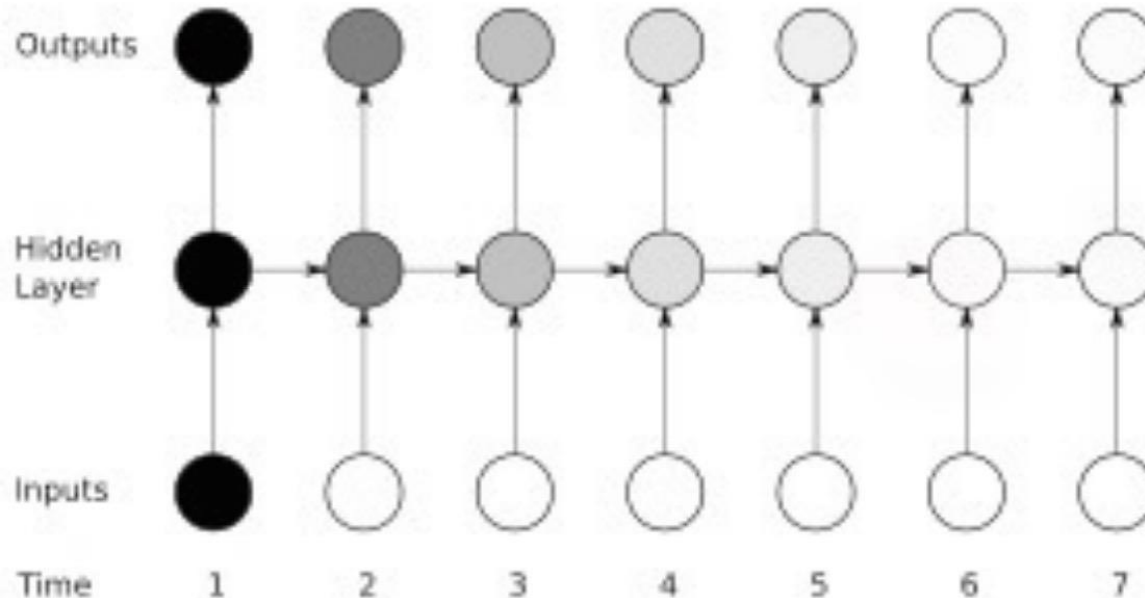
A: Temporal (time-based) vanishing gradients.



Long Short-Term Memory (LSTM)

Motivation:

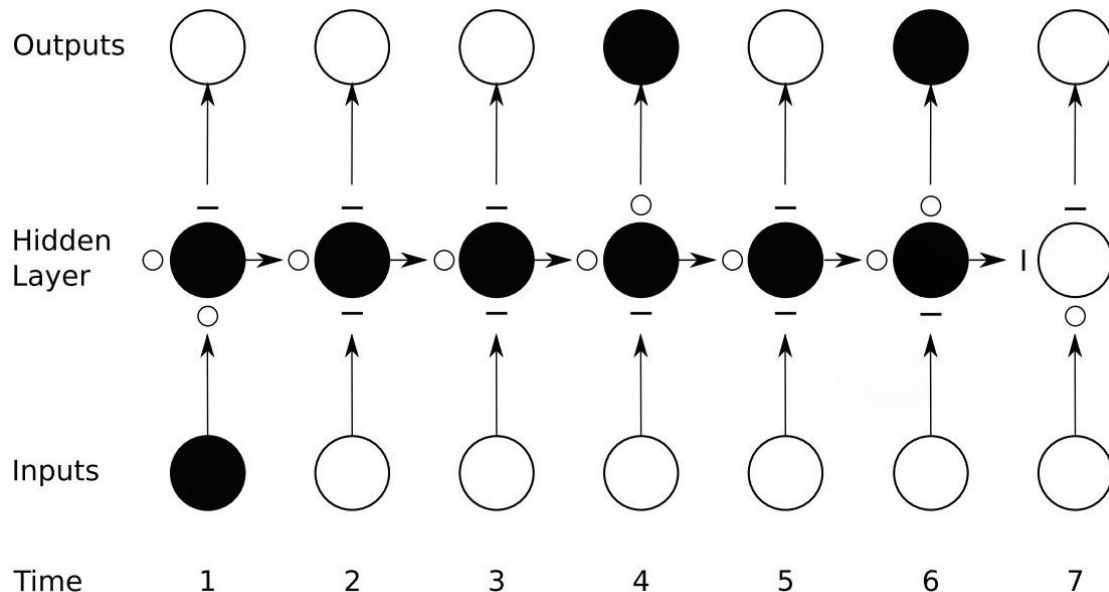
- Vanishing gradient problem for Standard RNNs
- Figure shows sensitivity (darker = more sensitive) to the input at time $t=1$



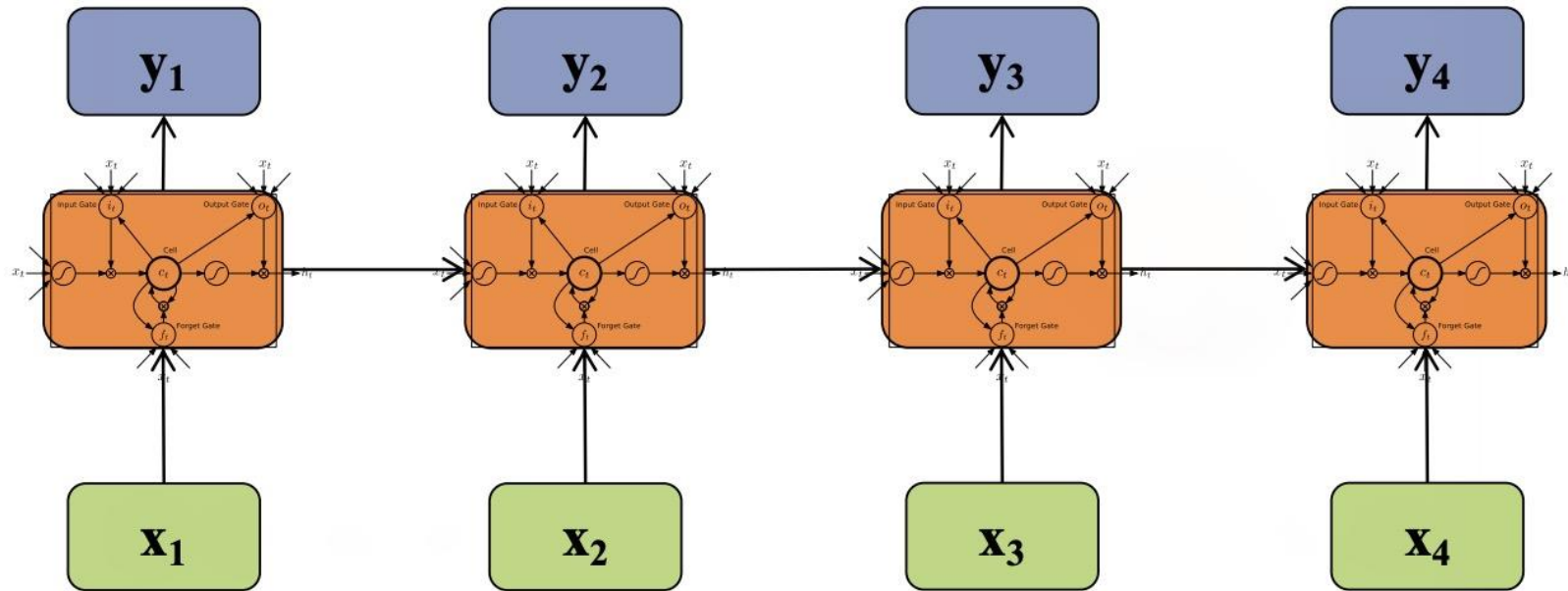
Long Short-Term Memory (LSTM)

Motivation:

- LSTM units have a rich internal structure
- The various “gates” determine the propagation of information and can choose to “remember” or “forget” information



Long Short-Term Memory (LSTM)



Long Short-Term Memory (LSTM)

Input gate: masks out the standard RNN inputs

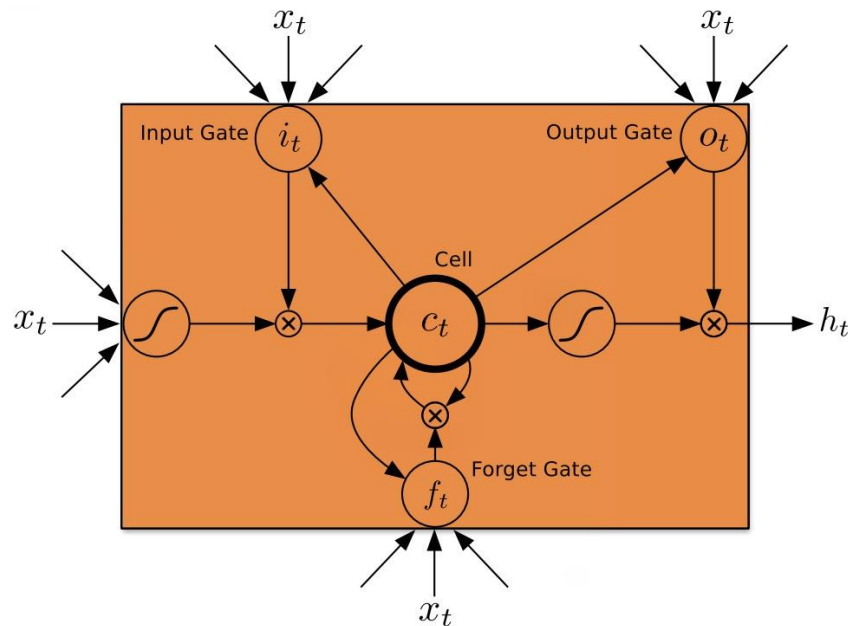
Forget gate: masks out the previous cell

Cell: stores the input/forget mixture

Output gate: masks out the values of the next hidden

Q: Which path is responsible for carrying long-term memory?

$$\begin{aligned}i_t &= \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i) \\f_t &= \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f) \\c_t &= f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c) \\o_t &= \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o) \\h_t &= o_t \tanh(c_t)\end{aligned}$$



Long Short-Term Memory (LSTM)

Input gate: masks out the standard RNN inputs

Forget gate: masks out the previous cell

Cell: stores the input/forget mixture

Output gate: masks out the values of the next hidden

Q: Which path is responsible for carrying long-term memory?

A: The cell state path c_t .

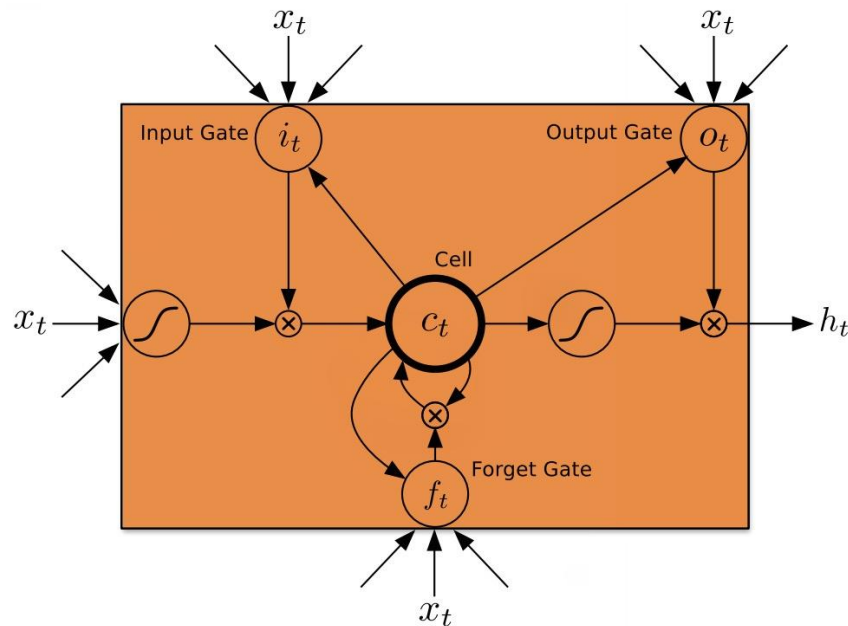
$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$



Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

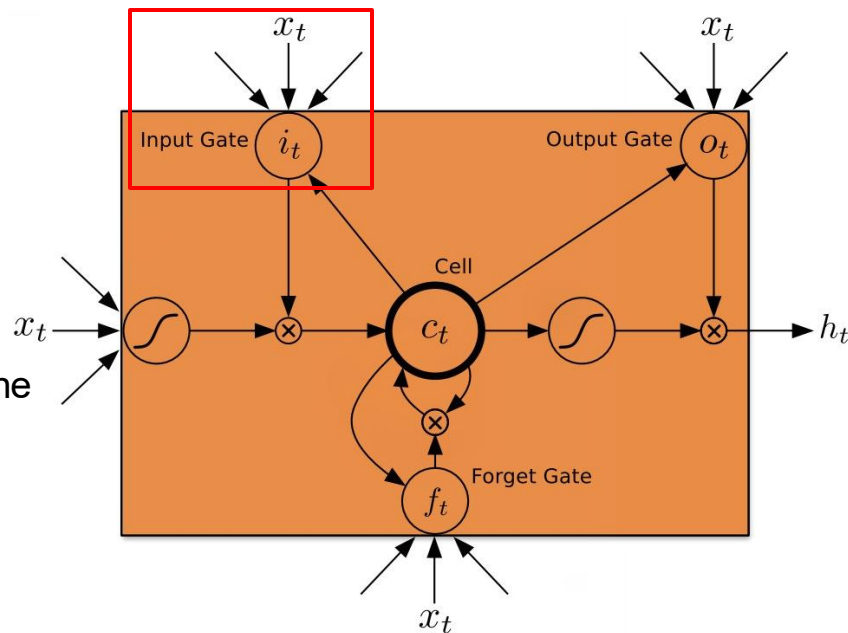
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Input Gate: Decides how much new information from the current input x_t should enter the cell state.



Q: What activation function is used for gates?

Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

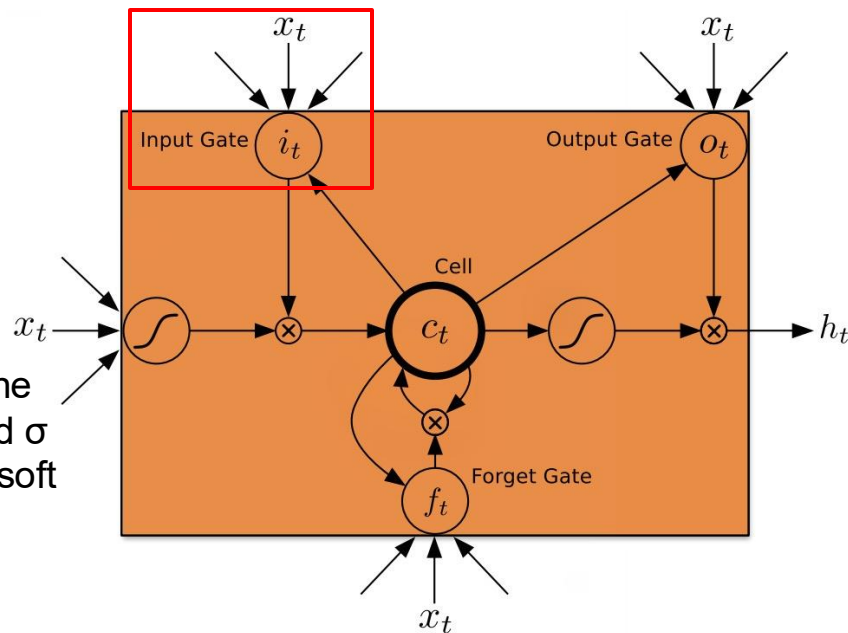
$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Input Gate: Decides how much new information from the current input x_t should enter the cell state. The sigmoid σ makes each component between 0 and 1, acting like a soft mask.

If $i_t \approx 1$: allow new information in

If $i_t \approx 0$: block new information



Q: What activation function is used for gates?

A: Sigmoid.

Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

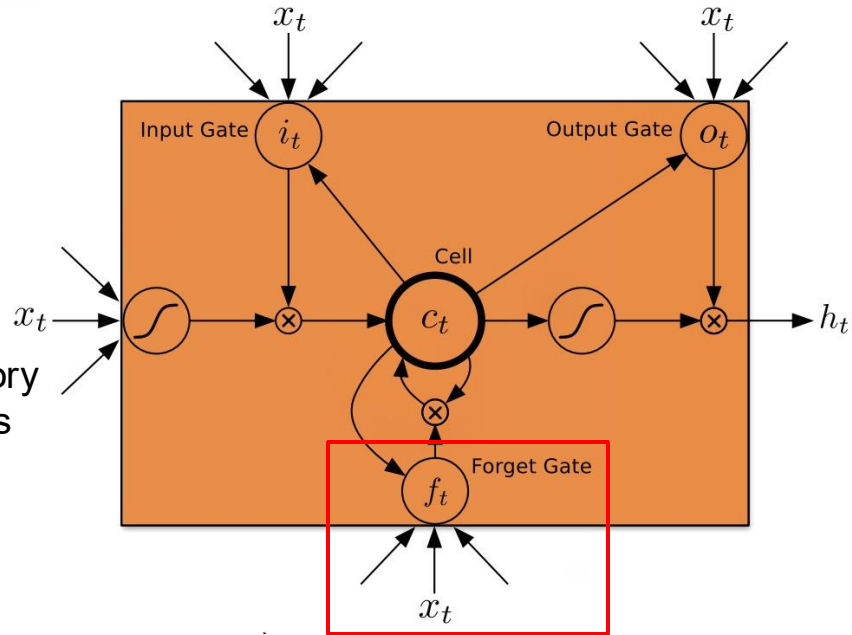
$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Forget Gate: Controls how much of the previous memory c_{t-1} should be forgotten. Also uses sigmoid $\rightarrow$ values between 0 and 1.

If $f_t \approx 1$: keep previous memory

If $f_t \approx 0$: forget previous memory



Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Cell State Update:

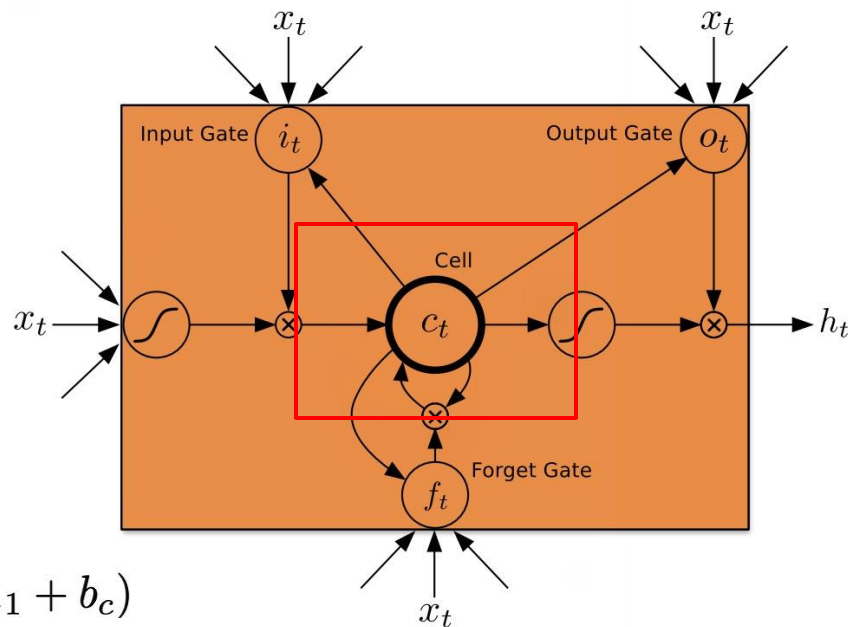
- Keep/Forget old memory:

$$f_t c_{t-1}$$

- Add/Not new candidate memory:

$$i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

- The cell state combines useful old memory and relevant new information.
- $\tanh$ ensures the new memory candidate is between -1 and 1.



Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

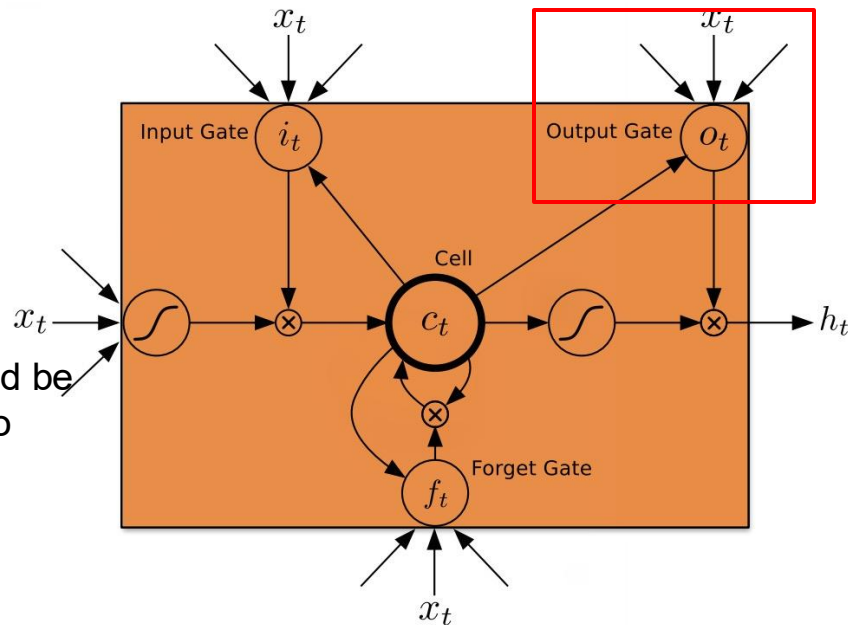
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Output Gate: Decides how much of the cell state should be exposed to the outside as hidden state. Uses sigmoid to control the “visibility” of the memory.



Q: Why not output the whole cell state directly?

Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

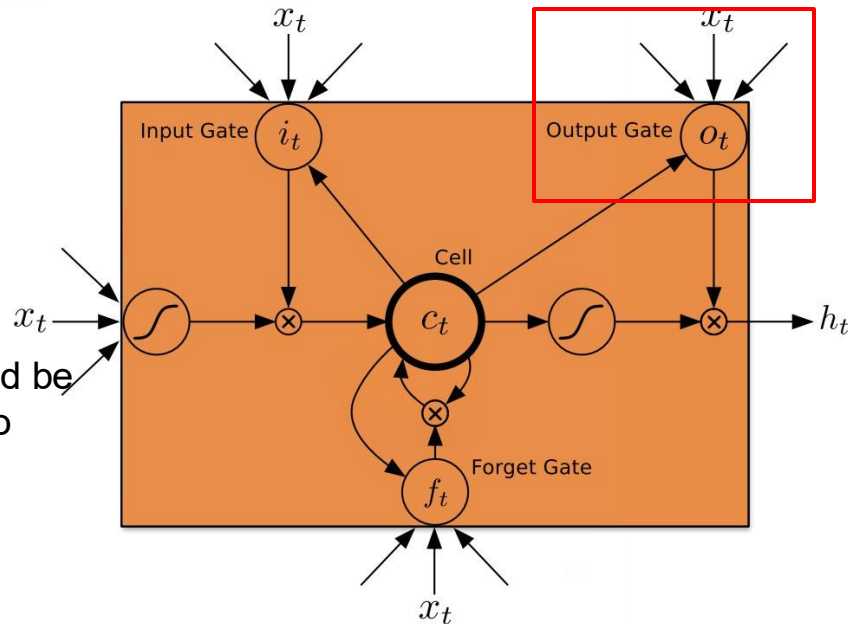
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Output Gate: Decides how much of the cell state should be exposed to the outside as hidden state. Uses sigmoid to control the “visibility” of the memory.



Q: Why not output the whole cell state directly?

A: Because not all memory is needed at each time step.

Long Short-Term Memory (LSTM)

$$i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + W_{ci}c_{t-1} + b_i)$$

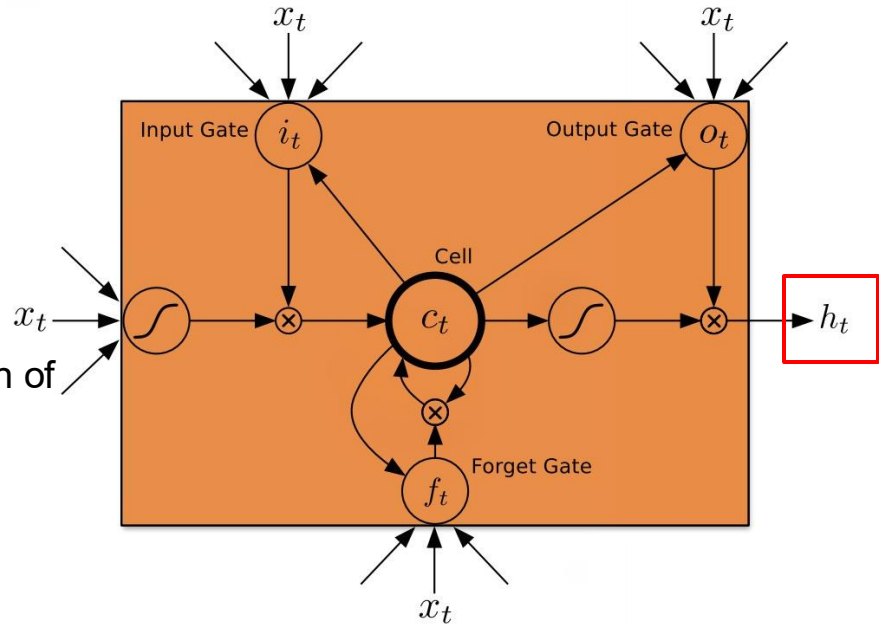
$$f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + W_{cf}c_{t-1} + b_f)$$

$$c_t = f_t c_{t-1} + i_t \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$$

$$o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + W_{co}c_t + b_o)$$

$$h_t = o_t \tanh(c_t)$$

Hidden State: The output gate o_t decides what portion of the cell state becomes the next hidden state.



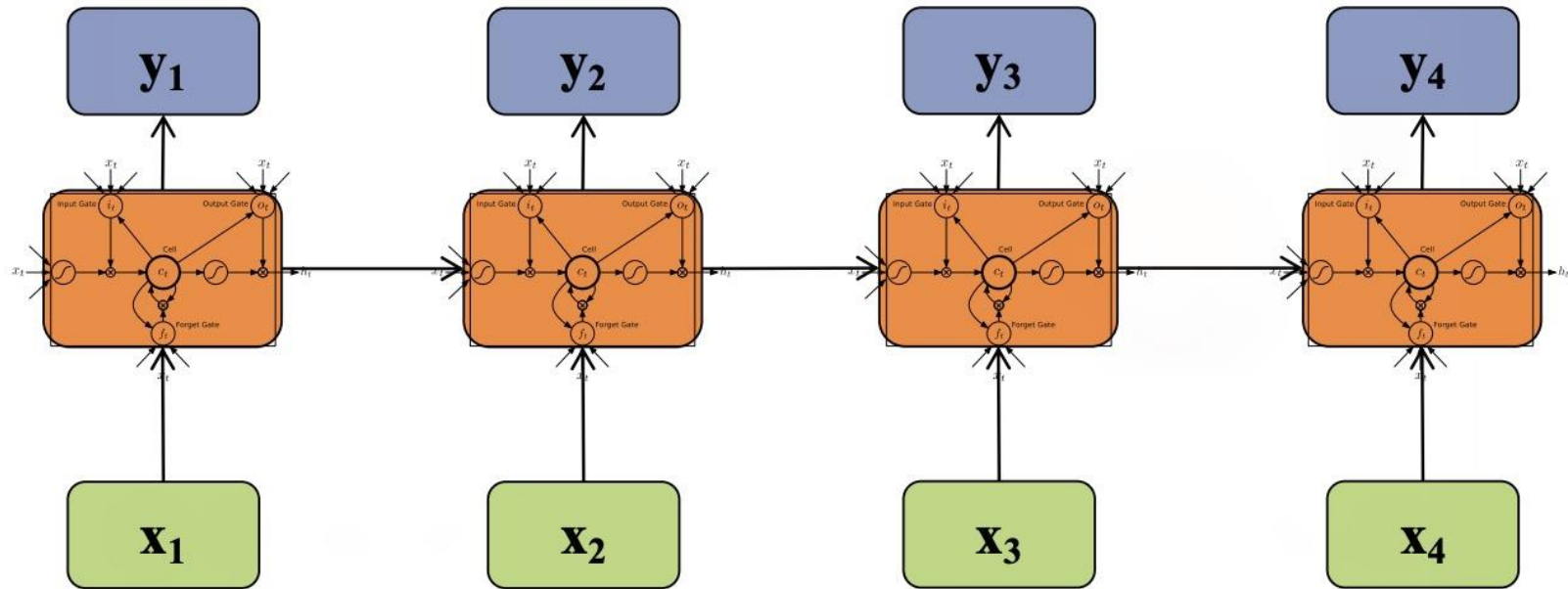
Visualised

https://www.reddit.com/r/learnmachinelearning/comments/smindilstm_visualized/

Long Short-Term Memory (LSTM)

Equation	Component	Role
$(i_t = \sigma(\dots))$	Input Gate	Controls how much new info enters memory
$(f_t = \sigma(\dots))$	Forget Gate	Controls how much past memory is kept
$(c_t = f_t c_{t-1} + i_t \tanh(\dots))$	Cell State	Updates long-term memory
$(o_t = \sigma(\dots))$	Output Gate	Controls what part of memory is output
$(h_t = o_t \tanh(c_t))$	Hidden State	The visible output of the LSTM

Long Short-Term Memory (LSTM)



How LSTM Prevents Vanishing Gradients (from large time steps)

How LSTM Prevents Vanishing Gradients (from large time steps)

Derivative:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

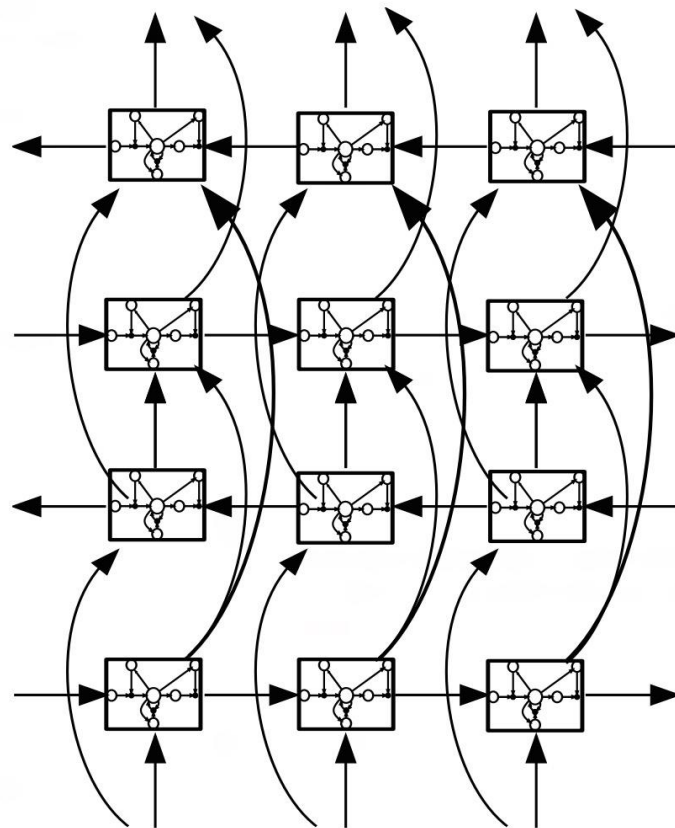
If $f_t \approx 1$:

- Gradient ≈ 1
- Memory lasts for hundreds of steps
- RNN remembers ~ 10 steps
- LSTM can remember > 500 steps

The key reason LSTMs avoid vanishing gradients is the near-linear cell state update. If gates produce values near 1, gradient flow remains stable over long sequences.”

Deep Bidirectional LSTM (DBLSTM)

- Same general structure as a Deep Bidirectional RNN, but with LSTM units in the hidden layers
- No additional representational power over DBRNN, but easier to learn in practice



Why not just use LSTMs for everything?

Everyone did, for a time.

But...

- They still have difficulty with long-range dependencies
- Their computation is inherently serial, so can't be easily parallelised on a GPU
- Gradient...large depth

Q: Why can't LSTMs be fully parallelized?

Why not just use LSTMs for everything?

Everyone did, for a time.

But...

- They still have difficulty with long-range dependencies
- Their computation is inherently serial, so can't be easily parallelised on a GPU
- Gradient...large depth

Q: Why can't LSTMs be fully parallelized?

A: Because each step depends on the previous hidden/cell state.

GRU (Gated Recurrent Network)

- Update gate
- Reset gate
- Candidate hidden state
- Final hidden state

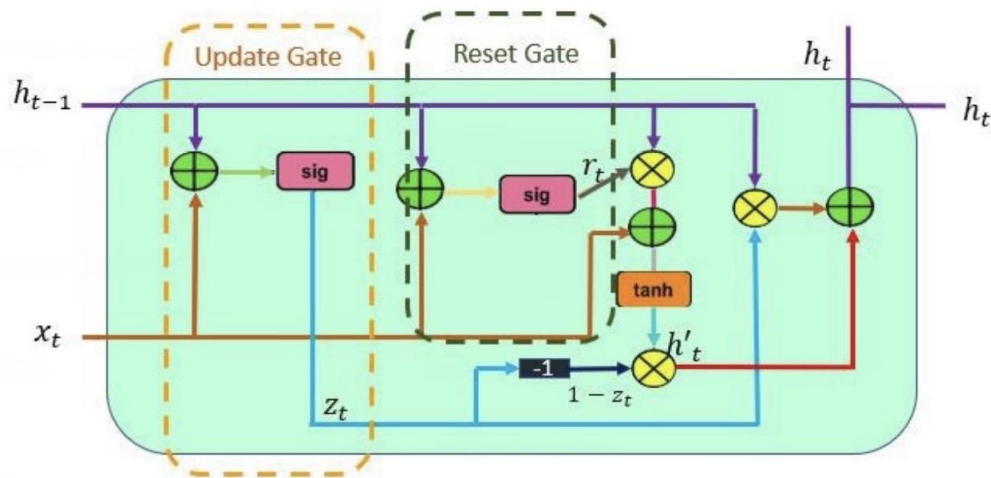
GRUs simplify LSTMs by removing the cell state and combining gates. The update gate mixes old and new memory. The reset gate decides how to incorporate past information.

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

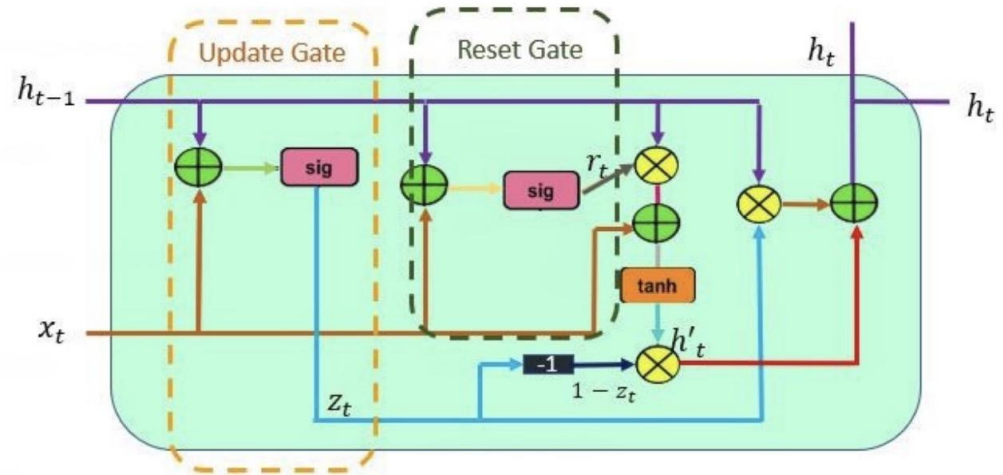
$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$



GRU (Gated Recurrent Network)

- Update gate

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

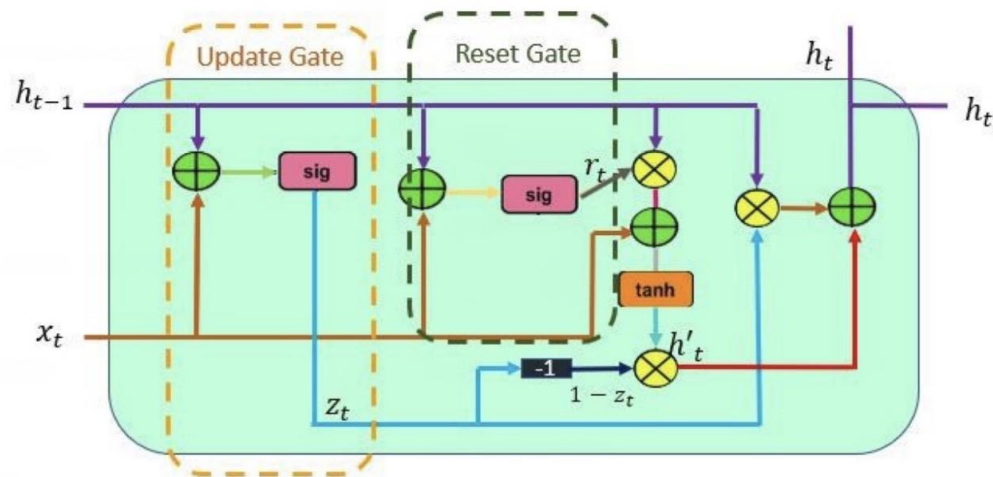


- Takes current input x_t and previous hidden state h_{t-1}
- Outputs values **between 0 and 1** (via sigmoid)
- Controls **how much past information to keep**

GRU (Gated Recurrent Network)

- Reset gate

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

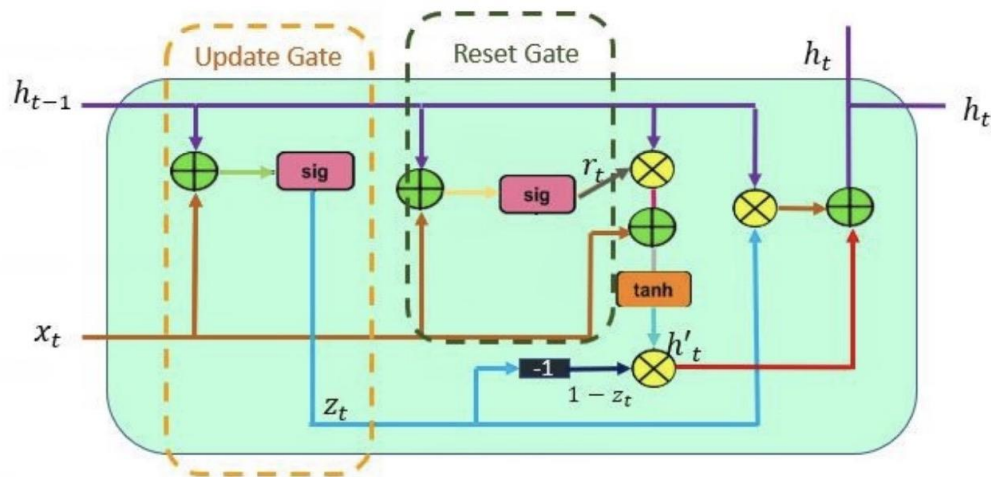


- Determines whether to **ignore** or **incorporate** the previous hidden state for generating the candidate memory.
- If $r_t \approx 0$: reset the memory $\rightarrow$ ignore past
- If $r_t \approx 1$: use past information as usual

GRU (Gated Recurrent Network)

- Candidate hidden state

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

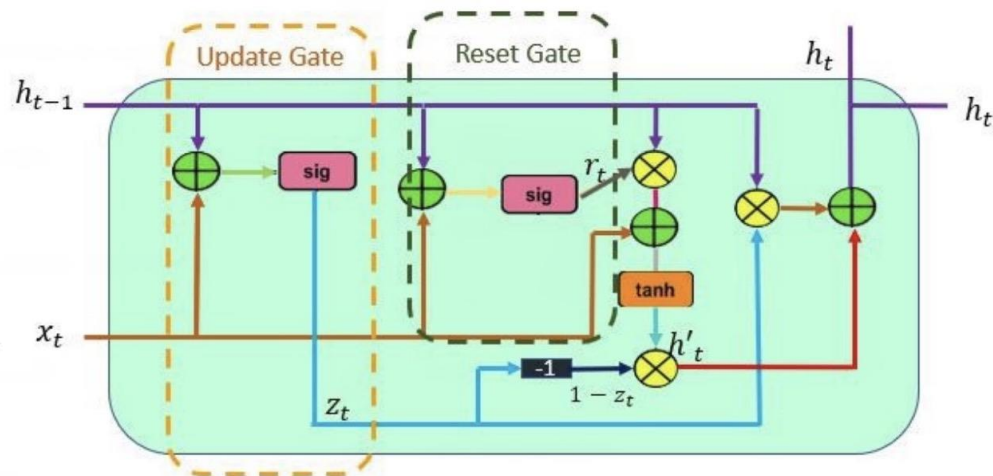


- Combines current input x_t and **reset-modified** past hidden state
- $r_t \odot h_{t-1}$ allows the GRU to selectively forget the past
- $\tanh$ ensures the candidate state is bounded between -1 and 1

GRU (Gated Recurrent Network)

- Final hidden state

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$



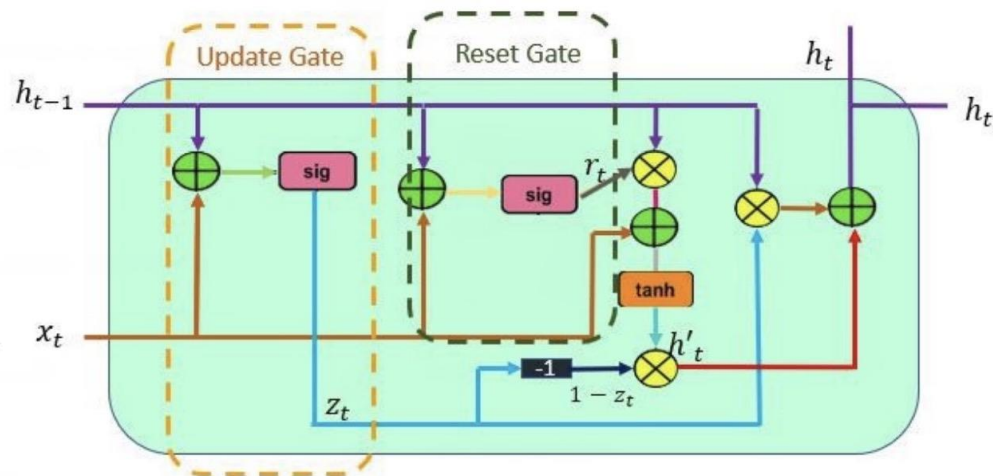
Q: Which gate in GRU controls forgetting and updating?

- This is an **interpolation** between:
 - The old hidden state h_{t-1}
 - The new candidate state $\tilde{h}_t$
- If $z_t \approx 1$: keep old memory → **copy hidden state**
- If $z_t \approx 0$: use new candidate → **update memory**

GRU (Gated Recurrent Network)

- Final hidden state

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

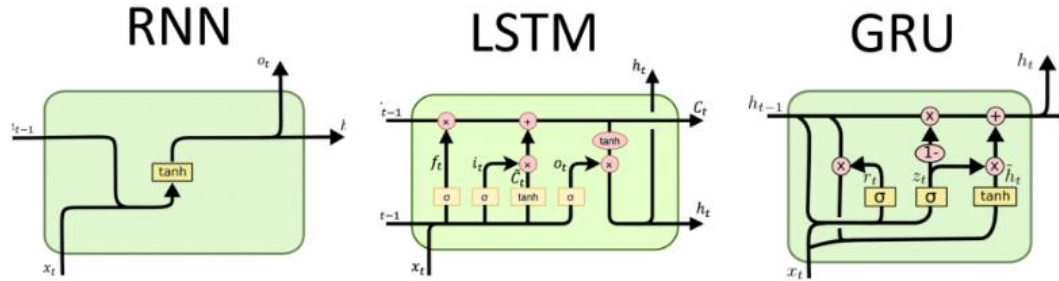


- This is an **interpolation** between:
 - The old hidden state h_{t-1}
 - The new candidate state $\tilde{h}_t$
- If $z_t \approx 1$: keep old memory → **copy hidden state**
- If $z_t \approx 0$: use new candidate → **update memory**

Q: Which gate in GRU controls forgetting and updating?

A: Update gate.

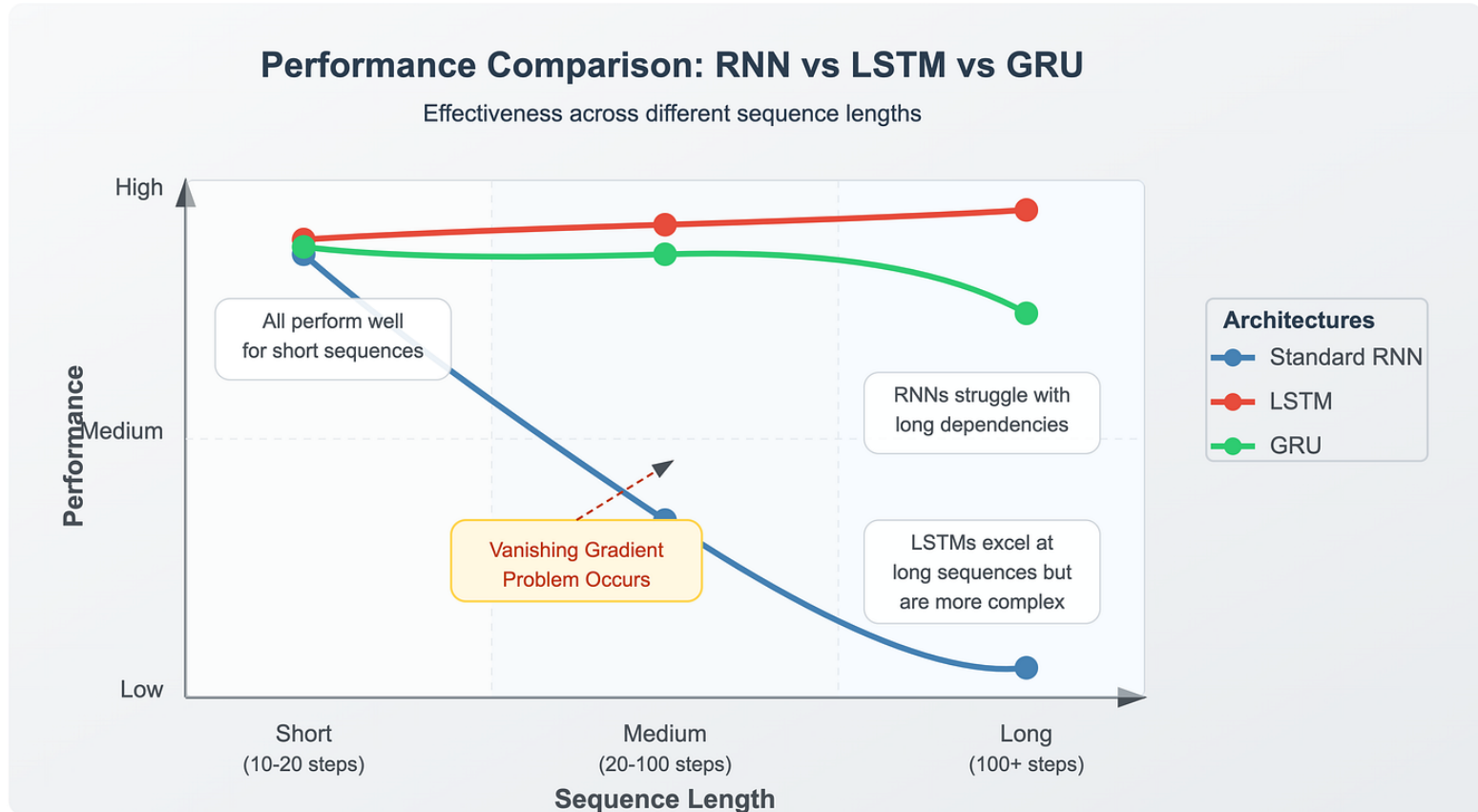
GRU (Gated Recurrent Network)



GRU (Gated Recurrent Units) are similar to the LSTM networks. GRU is a kind of newer version of RNN. However, there are some differences between GRU and LSTM.

- GRU doesn't contain a cell state
- GRU uses its hidden states to transport information
- It Contains only 2 gates(Reset and Update Gate)
- GRU is faster than LSTM
- GRU has lesser tensor's operation that makes it faster

Standard RNN, LSTM, GRU



Standard RNN, LSTM, GRU

Feature	Standard RNN	LSTM	GRU
Number of Gates	0	3 (Input, Forget, Output)	2 (Update, Reset)
Memory Mechanism	Hidden state only	Hidden state + Cell state	Hidden state only (no cell state)
Equations Complexity	Simple	Most complex	Medium
Ability to Capture Long-Range Dependencies	✗ Poor (vanishing gradients)	✓ Excellent (cell state, forget gate)	✓ Very good (update gate)
Computational Cost	Low	Highest	Medium (≈ 25–30% faster vs LSTM)
Parameters	Fewest	Most	Fewer than LSTM
Training Speed	Fast	Slowest	Fast (close to RNN)
Ease of Implementation	Easiest	Hardest	Moderate
Risk of Overfitting	Lower	Higher	Medium
Best Use Cases	Short sequences, simple tasks	Complex sequences, NLP, speech	Real-time systems, less training time
Handles Vanishing Gradients (Temporal)	✗ No	✓ Yes (Constant Error Carousel)	✓ Yes (update gate)
Handles Vanishing Gradients (Depth)	✗ No	✗ No	✗ No
Modeling Flexibility	Low	High	Medium
Memory Control	None	Very explicit control	Moderately explicit
Performance on NLP Benchmarks	Weak	Strong	Comparable or slightly lower than LSTM
Interpretability	Simple dynamics	Clear gate roles	Clear gate roles

Comparison of the three architectures

LSTM in PyTorch

```
# Hyper-parameters
input_size  = 10    # dimension of  $x_t$ 
hidden_size = 20    # dimension of  $h_t$ 
num_layers  = 1     # single-layer LSTM
seq_len     = 5     # sequence length
batch_size  = 3

# LSTM layer
lstm = nn.LSTM(input_size=input_size,
                hidden_size=hidden_size,
                num_layers=num_layers,
                batch_first=True)  # input shape: (batch, seq, feature)
```

GRU in PyTorch

```
# Hyper-parameters
```

```
input_size  = 10
```

```
hidden_size = 20
```

```
num_layers  = 1
```

```
seq_len     = 5
```

```
batch_size  = 3
```

```
# GRU layer
```

```
gru = nn.GRU(input_size=input_size,  
              hidden_size=hidden_size,  
              num_layers=num_layers,  
              batch_first=True)
```

Thanks
Q&A

Next week: Transformer